

The biomimetic pipeline — measured enamel → printable lattice

The problem it solves: the measured morphometrics and the parametric lattice computer-aided-design (CAD) model existed *separately* — the CAD was driven by **hand-picked parameters**. **This pipeline makes the biology drive the geometry.**

UPSTREAM • the measurement stack

microct_pipeline/ — upstream measurement repository

X-ray micro-computed-tomography (micro-CT) stack → 101 slices → `detect_rods.py` (watershed) → `track_rods_piv.py` (particle-image-velocimetry optical-flow tracking → three-dimensional rod tracks; pitch / yaw / tilt)
→ `compute_tortuosity.py` → `analysis_decussation.py` (Fourier band periodicity, k -means angle clustering) → `som_approach/` (self-organising-map band segmentation)

STAGE 0 • INGEST

ingest/ — six ingesters, adapter pattern; one per upstream tool

`hsb_ | piv_ | smoothed3d_ | som_ | tracking_ | validation_ingester` → `merge.py`
Each adapter knows *one* upstream format. Adding a new instrument = adding one ingester — nothing else changes.

`morphometrics.json` CONTRACT 1 —
`jsonschema.validate` on load (`merge.py`)
required: `schema_version` · `specimen_id` · `units` · `provenance` · `depth_profiles` · `angle_stats` · `bands`

STAGE 1 • MAPPING — biology becomes geometry

mapping/ (this is the intellectual core)

pitch profile → **RING_ROTATION**: integrate signed pitch, $\phi(z) = \int p(z) dz$; classify against linear / accelerating / sigmoid, lowest root-mean-square error. If no shape fits within 10%: **TWIST_TYPE** = **measured** + full (z, ϕ) spline.

self-organising-map bands → **bridge geometry**: $N_{\text{bridge}} = \text{clip}(\text{round}(H/\lambda), 2, 8)$; **RING_ROTATION**[0] = band-0 direction; **CENTER_SPACING** = mean band width

rod morphology → **ROD_DIAMETER**, **ROD_TAPER_FACTOR**; >15% non-monotone → **per_ring_diameter** + *radially-graded* generator

tortuosity → **twist_perturbation_amplitude_deg** (clipped $[0, 15]^\circ$; drives the hierarchical-twist generator)

SCALE + CLAMP: real rod $\approx 4 \mu\text{m}$; printed rod = 2 mm ⇒ **biology_scale_factor** = $500\times$. Every feature clamped to **sla_min_feature_mm** = 0.3 mm — and **every clamp is logged with its raw AND clamped value** into **provenance.scale_clamps**. **The mapping is auditable.**

4 LATTICE INVARIANTS enforced HERE — bridge/rod ratio, lattice density, junction-sphere plate clearance, taper $\in [0, 1]$ — so no downstream stage ever fails validation.

`cad_parameters.json` CONTRACT 2 —
`jsonschema.validate` on load (`feature_to_cad`)
stock CAD keys verbatim (ROD_DIAMETER, CENTER_SPACING,
RING_ROTATION, TWIST_TYPE, ...) + new-model extensions

STAGE 2 • CAD + MESH

generators/ — runs in *cad_env*, then *sfepy_env*

`cad_runner.py` → `stock lattice_cad.py` (CadQuery) 2a. `cad_integrity.py` ← integrity check *post-CAD, pre-mesh* `mesh_runner.py` → `stock lattice_mesh.py` (Gmsh)

NON-INVASIVE: nothing inside `cad_modeling/` is ever modified. Stock tools are invoked through **conda run** with their existing command-line contracts intact. **Two separate conda environments because the dependency stacks genuinely conflict.**

STAGE 3 • FINITE-ELEMENT ANALYSIS (FEA) — the strain solver

fea/strain_solver.py

The stock solver prescribes a DISPLACEMENT. This pipeline does not. It solves for the displacement that produces a target stress: seed d_0 → run FEA → read average von-Mises stress → if $|VM - 200|/200 \leq \text{tol}$, accept; else $d \leftarrow d \cdot (200/VM)$ and re-run. **The mesh is held fixed** — only the displacement changes.

WHY (this is the answer they want): every design is then evaluated at **the same load state** ($\approx 200 \text{ MPa}$). It decouples objective quality from incidental stiffness differences — otherwise a stiffer lattice would “win” for reasons that have nothing to do with the objective. Records **critical_strain_at_200MPa** as a first-class metric.

STAGES 4–5 • METRICS → PLUGGABLE OBJECTIVE

metrics/ • *objectives/registry.py*

crack deflection: rasterise per-element principal stress onto a 32^3 grid → gradient field → seed **64 streamlines** on the bottom face → integrate to the top → report mean / 90th-percentile / max of arc-length ÷ chord-length. **biomimicry score**: z -scored Euclidean distance across five measured-vs-generated feature pairs. Smaller = closer to the real biology.

Every configuration file under `config/objectives/` becomes a loadable objective with `.score(metrics)` and a direction. Five ship: **crack_deflection** (default) · **toughness** · **stiffness_target** · **biomimicry_score** · **composite**. A **stress_target** block drives the strain solver, so **every objective is load-normalised by construction**.

STAGE 6 • ORCHESTRATION — four modes, one core

orchestration/

`pipeline.py` one run | `sweep.py` parametric sweep | `optimize.py` Optuna Bayesian optimisation | `closed_loop.py` ← the clever one

THE CLOSED LOOP — and this is the punchline of the whole system: run N trials, take the top- K by objective, then **INVERT** the mapping back into morphometric space and write `biomimicry_targets.json` — the rod diameters, band widths, wavelengths and directions **to go look for in a real specimen**. The simulation tells the microscope what to hunt for. A biology-informed *wanted poster*.

The engineering — and the honest boundary

When a panelist asks “why did you build it that way?” — the answer is never “faster” and never “cheaper”. Every decision below is about **isolation, contracts, auditability, or an honest signal**.

THE SIX DESIGN DECISIONS

each one is a question you will be asked

- 1. Non-invasive wrapping** — *not* “subprocesses are faster”. Nothing inside `cad_modeling/` is ever modified. The stock CadQuery, Gmsh, and SfePy tools are invoked through `conda run` with their existing command-line contracts intact; the pipeline auto-generates a driver that imports the stock module *as a library* so it keeps full parameter control (including `RING_ROTATION`, which the command line does not even expose). *The reason is quarantine: the upstream research code keeps working, unchanged, while the pipeline wraps it.*
- 2. Three canonical schemas — a contract, not a convention.** Every stage hand-off is governed by a JSON Schema: `morphometrics` (ingest→mapping), `cad_parameters` (mapping→CAD), `objective` (config→scoring). *A stage boundary you can machine-check is a boundary that cannot silently drift — and `tests/test_value_carrythrough.py` exists precisely to prove a value survives the whole chain.*
- 3. Two conda environments.** CAD runs in `cad_env`; mesh, FEA and metrics run in `sfePy_env`. *Not a performance choice — the dependency stacks genuinely conflict. Isolation is the point.*
- 4. The load-normalised operating point.** The strain solver bisects displacement until representative stress hits ≈ 200 MPa, then reports `critical_strain_at_200MPa`. *Without it, a stiffer lattice “wins” the objective for reasons that have nothing to do with the objective. This is the single most important scientific decision in the pipeline — lead with it.*
- 5. Auditable clamps.** Biology is $\sim 4\ \mu\text{m}$; the printer floor is 0.3 mm. Every manufacturability clamp is logged **with both the raw and the clamped value** into `provenance.scale_clamps`. *You can always reconstruct what the biology said before the printer overruled it. A clamp that isn’t logged is a lie.*
- 6. Invariants enforced at MAPPING time, not at failure time.** Four lattice invariants (bridge/rod ratio, lattice density, junction-sphere plate clearance, taper bounds) are enforced in the mapping layer, so CAD and meshing never receive a geometry they cannot build. *Fail early, in the layer that can explain why.*

VERIFICATION — `python tests/run_all.py`

Five modules, and **none of them needs `cad_env` or `sfePy_env`** — the tests run anywhere, on a clean machine, in seconds.

- `test_strain_solver` — linear FEA converges in one iteration; mild non-linearity within five
- `test_crack_deflection` — **known-answer test**: a uniform field must give tortuosity ≈ 1.0 ; a radial bowl must bend paths into `[1, 2]`
- `test_mapping` — monotone pitch $\Rightarrow$ monotone rotation; **clamps are logged**; eccentricity $> 0.2 \Rightarrow$ ellipse
- `test_new_models` — twist = 0 at $z = 0$ and total rotation at $z = H$
- `test_objective_registry` — all five objectives load; the default is monotone in tortuosity

*The crack-deflection test is the one to name out loud: it is a **known-answer test** against a field whose right answer is analytic.*

THE HONEST BOUNDARY (from your own doc — say it before they ask)

- The stock meshing chain still throws a piecewise-linear-complex error on some tight geometries. **That is upstream of the pipeline**; the sphere-aware clamp mitigates the common case but does not fix the root cause.
- The radially-graded generator uses the **mean** of the per-ring diameters. True per-ring variation requires replacing the hexagonal-grid generator.
- The **composite** objective’s weights are **illustrative**. They must be cross-validated against a held-out specimen before anyone uses them for production optimisation.
- No fatigue, no cyclic loading.** The stress bisection assumes **monotonic compression in the linear-elastic regime**. That is the modelling boundary, and it is a real one.

HOW TO TELL THIS IN 60 SECONDS — to people who will never see the code

“We image real tooth enamel with X-ray micro-computed-tomography and measure how the rods actually weave — their diameter, their twist, the banding. Those measurements used to sit in one pile, and the parametric lattice we three-dimensionally print sat in another, driven by numbers a person picked by hand.

*I built the pipeline that connects them. Measured morphometrics go in; a validated contract file comes out; a mapping layer turns biology into geometry — scaled five-hundred-fold to printable size, with every manufacturability clamp logged so you can always see what the biology said before the printer overruled it. Then it meshes, runs finite-element analysis, and here is the part I care about: it does not prescribe a displacement, it **solves** for the displacement that puts every single design at the **same 200-megapascal load state** — so when one lattice scores better on crack deflection, that is the geometry talking, not incidental stiffness.*

*And then it runs backwards. Take the best-performing lattices, invert the mapping, and it tells you which rod diameters and band widths **to go look for in a real specimen**. The simulation tells the microscope what to hunt for.”*

TRUST MECHANICS — where each guarantee is produced (read from source)

Hashes: **INGEST** — `merge.py` SHA-256s every source (`run_context.hash_existing`) $\rightarrow$ `provenance.source_hashes`; **MAPPING** — `feature_to_cad.py` stamps `morphometrics_sha256` into the CAD params (output $\leftrightarrow$ its exact input). **Validation:** *both* contracts — `jsonschema.validate` on load (`merge.py` / `feature_to_cad.py`); structure can’t silently drift.

Clamps: **MAPPING** — `scale.py` logs every clamp **raw + clamped** $\rightarrow$ `provenance.scale_clamps`. **Determinism:** the core **CAD** $\rightarrow$ **mesh** $\rightarrow$ **FEA** $\rightarrow$ **metrics** path has **no random seed**; the only stochastic step is the search (`TPESampler(seed=42)`), so even that is reproducible. **Honest gap:** hashes are **recorded, not re-verified** $\rightarrow$ integrity-check upgrade = re-hash + compare + fail on mismatch.